

Semantic retrieval & RAG (Retrieval Augmented Generation)

Semantic retrieval and question answering

Maciej Piasecki, Konrad Wojtasik

Katedra Sztucznej Inteligencji, Grupa Naukowa G4.19

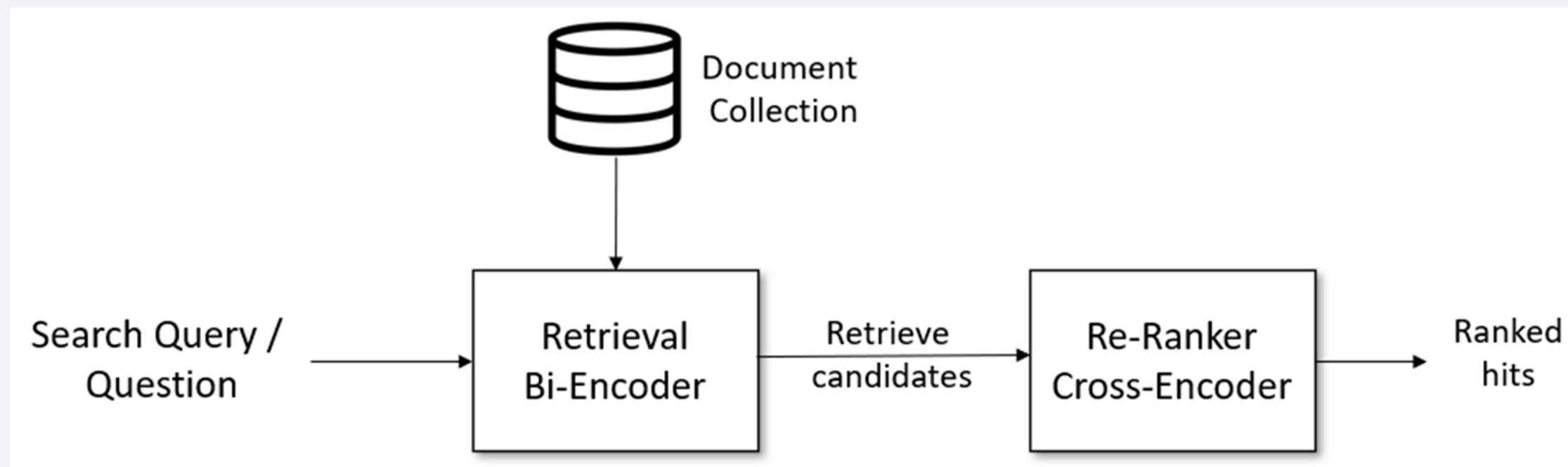
CLARIN-PL

clarin-pl.eu

Semantic retrieval

- Goal:
 - „to generate an ordered list of texts retrieved from a corpus in response to a query for a particular task“,
 - Query-document matching function.
- Problems:
 - Representation: query vs document,
 - Matching methods,
 - Process efficiency for very large collections.

Semantic retrieval



<https://www.sbert.net/examples/applications/information-retrieval/README.html>

Text Similarity Ranking: Problems

- Question Answering (QA)
 - Information Retrieval – short queries
 - Semantic Retrieval
 - Question answering – queries similar to natural language questions.
- Community Question Answering (CQA)
 - search in question and answer databases, e.g. Quora or Stack Overflow, „locating another user who has asked the same or similar question“,
 - frequently-asked questions (FAQ).
- Information Filtering
 - Queries expresses an information need.
- Text Recommendation
 - suggesting texts (text objects) based on similarity.

Semantic retrieval: schema

- Lexical retrieval
 - based on inverted index)
 - from terms to documents,
 - keyword retrieval
 - IR: binary search model
- Ranking based on term occurrences (global and local).
- Re-ranking
 - for the k most suitable
 - Machine learning-based ranking adjustment (for the top k only)
- Extracting the answers:
 - passage retrieval,
 - answer extraction.

Lexical retrieval

- *Exact term matching:*
 - terms (query) vs terms (document).
- Matching/ranking measures, e.g. BM25

$$\text{BM25}(q, d) = \sum_{t \in q \cap d} \log \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} \cdot \frac{\text{tf}(t, d) \cdot (k_1 + 1)}{\text{tf}(t, d) + k_1 \cdot (1 - b + b \cdot \frac{l_d}{L})}$$

- $\text{df}(t)$ – number of documents containing t
- N – collection size
- $\text{tf}(t, d)$ – t frequency in document d
- l_d – length of d
- L – average document length in the collection
- k_1 and b – parameters.

Measures for assessing relevance rankings

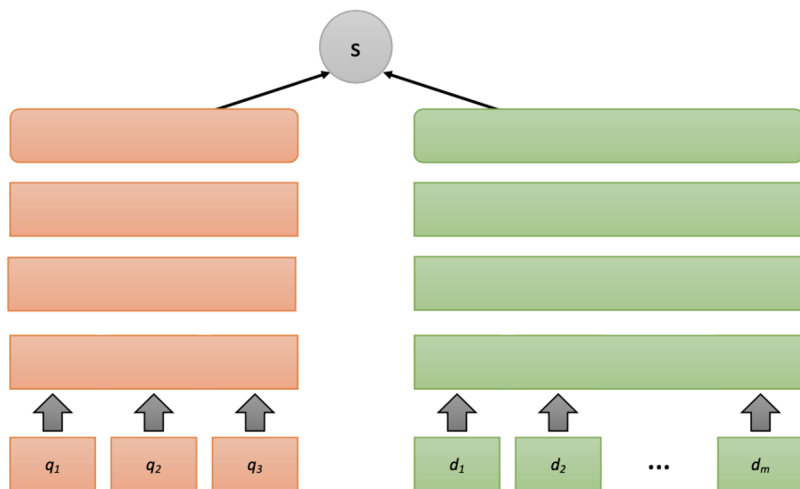
- Cut-off precision and recall:
 - to the first k positions of the generated ranking, most often recall@100
 - MAP – Mean Average Precision,
 - MRR – Mean Reciprocal Rank (Wikipedia)

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

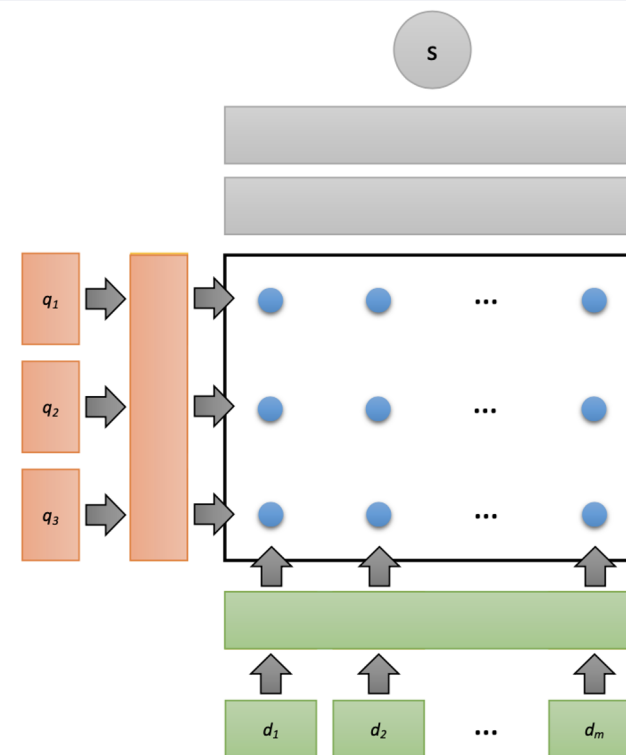
- Normalized Discounted Cumulative Gain (nDCG) (Lin, et al. 2021)
 - assumes a finite range of values of the relevance measure $rel()$, IDCG – ideal value of DCG

$$\text{DCG}(R, q) = \sum_{(i,d) \in R} \frac{2^{\text{rel}(q,d)} - 1}{\log_2(i + 1)} \quad \text{nDCG}(R, q) = \frac{\text{DCG}(R, q)}{\text{IDCG}(R, q)}$$

Query-document interaction models



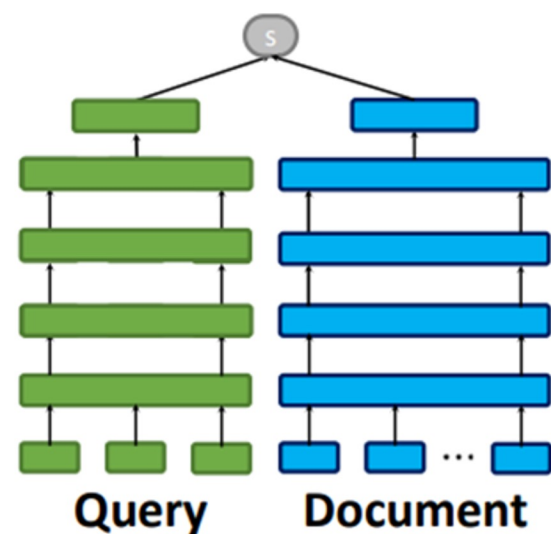
(a) a generic representation-based neural ranking model



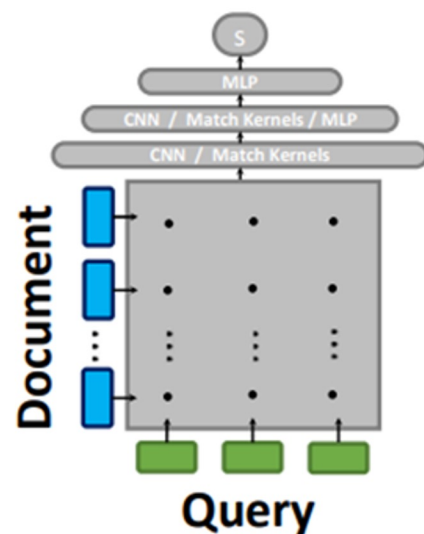
(b) a generic interaction-based neural ranking model

- Representation based models vs Interaction based models.

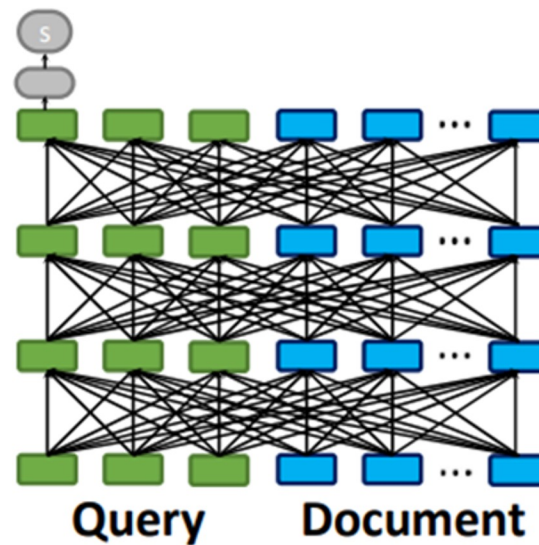
Query-document interaction models



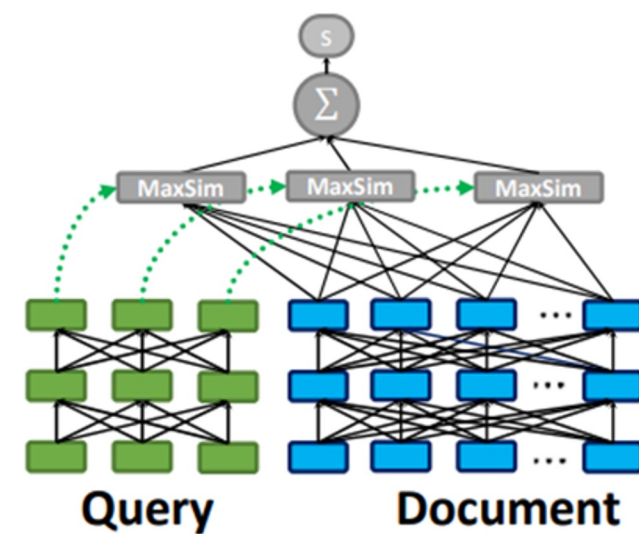
(a) Representation-based Similarity
(e.g., DSSM, SNRM)



(b) Query-Document Interaction
(e.g., DRMM, KNRM, Conv-KNRM)



(c) All-to-all Interaction
(e.g., BERT)



(d) Late Interaction
(i.e., the proposed ColBERT)

ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT

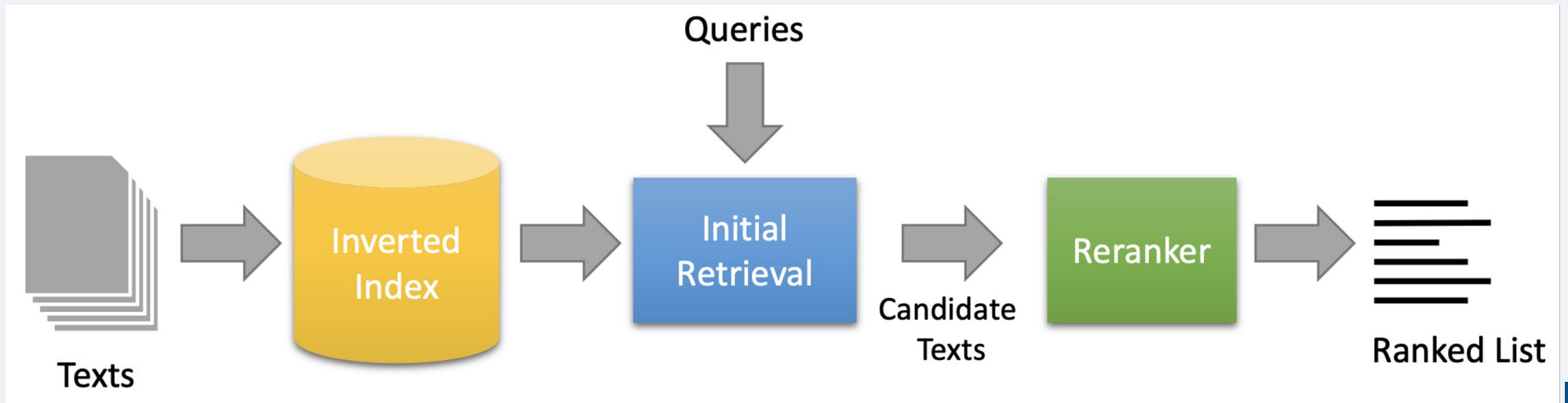
monoBERT – idea

- Re-ranking based directly on BERT.

- Task:

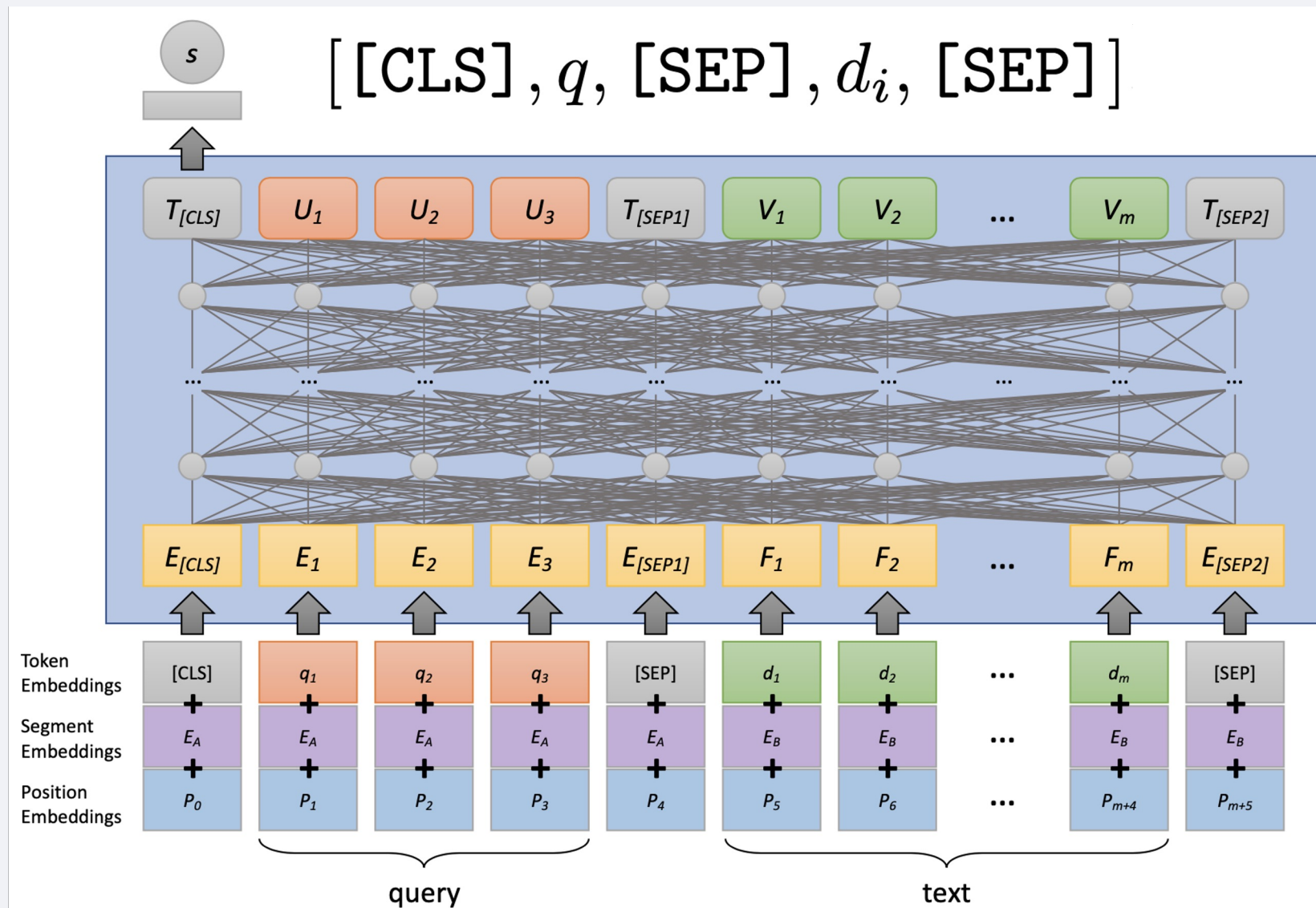
$$P(\text{Relevant} = 1 | d_i, q)$$

- Architecture:

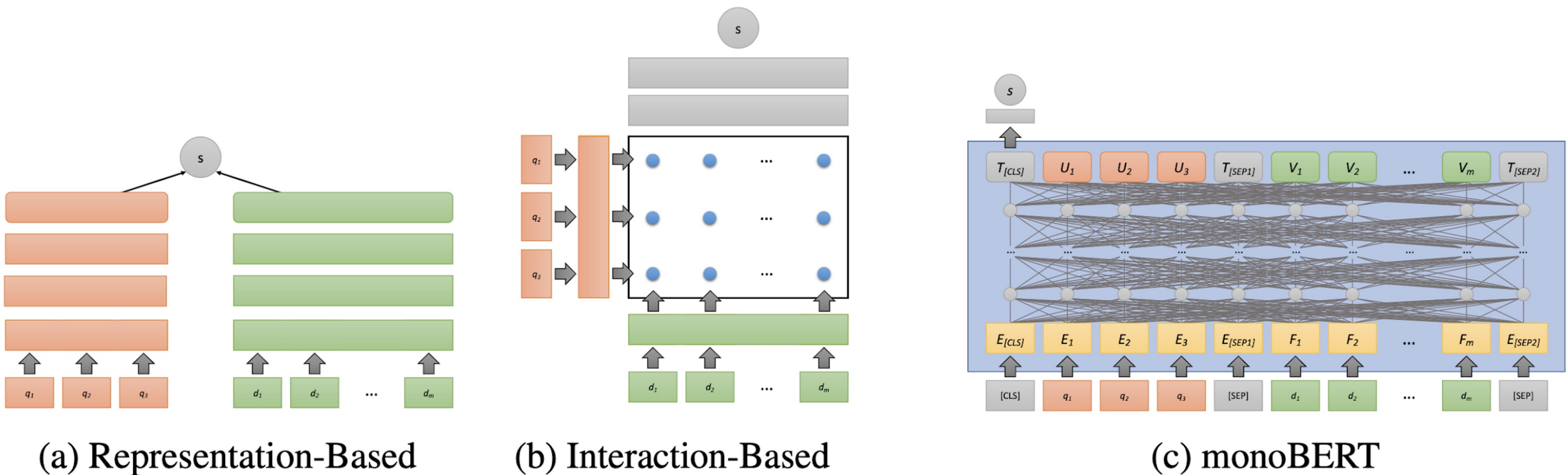


monoBERT – schema

- *Cross-encoder*

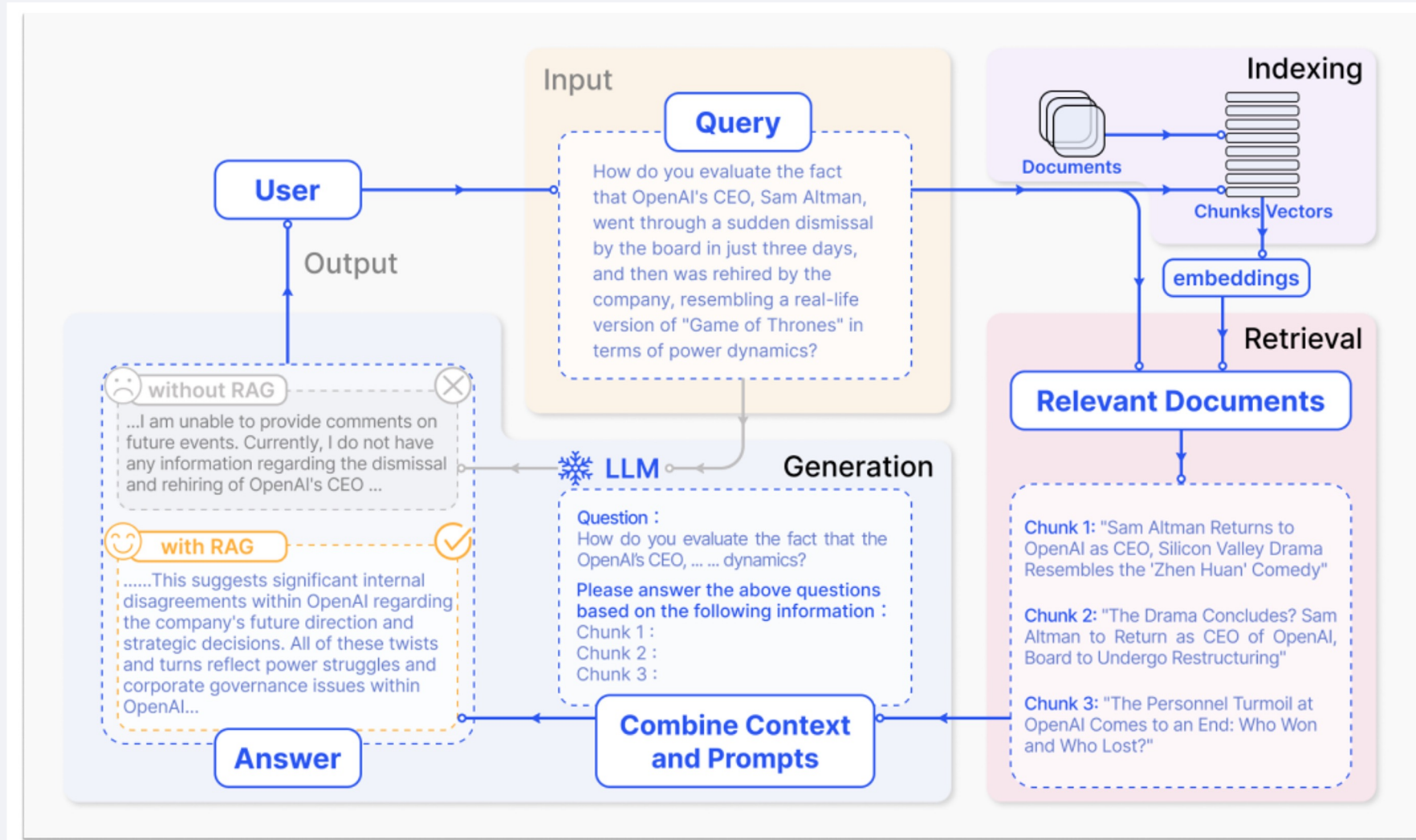


monoBERT vs neural ranking models



- Interactive models show higher scores
- Attention-modeled Q&A token interactions
- „Attention appears to serve as a one-size-fits-all approach to extracting signal from term interactions, replacing the various techniques used by pre-BERT interaction-based models” (Lin et al., 2021)

Retrieval Augmented Generation (RAG)



Retrieval Augmented Generation (RAG)

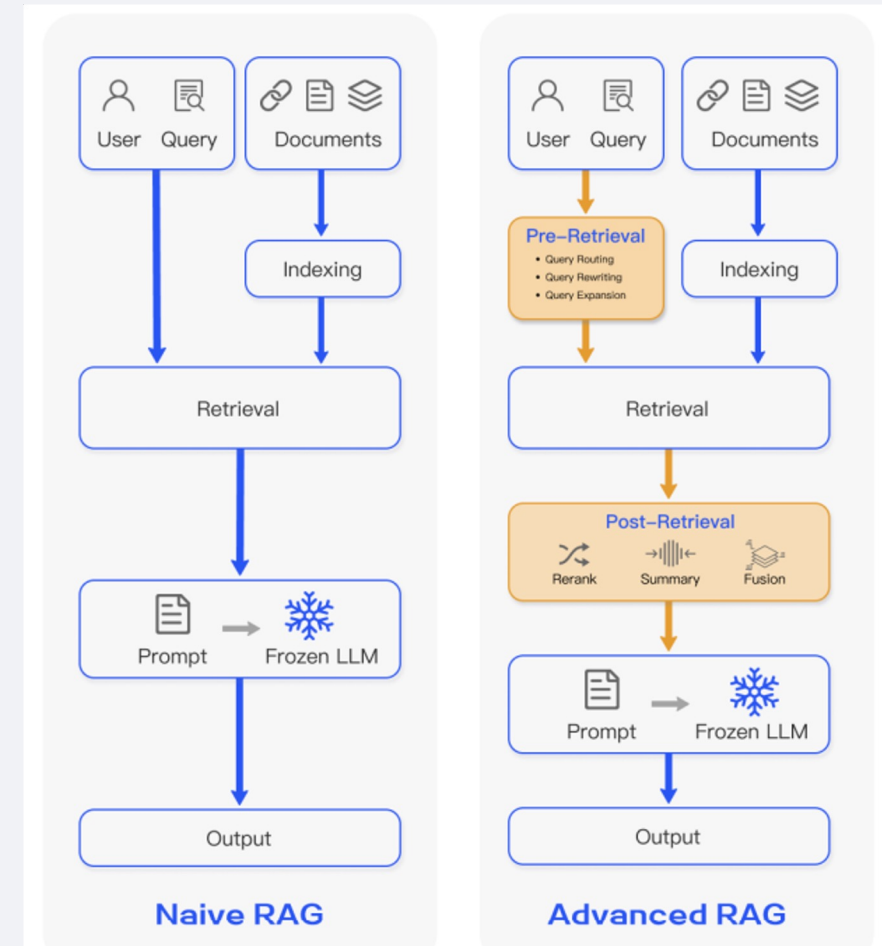
Advantages of RAG:

- allows for delivering the latest information to the model
- reduces hallucinations
- allows for using LLM as an assistant in a specific field
(company documents, descriptions of the latest events)
- enables better interpretation of the response and indication of the source

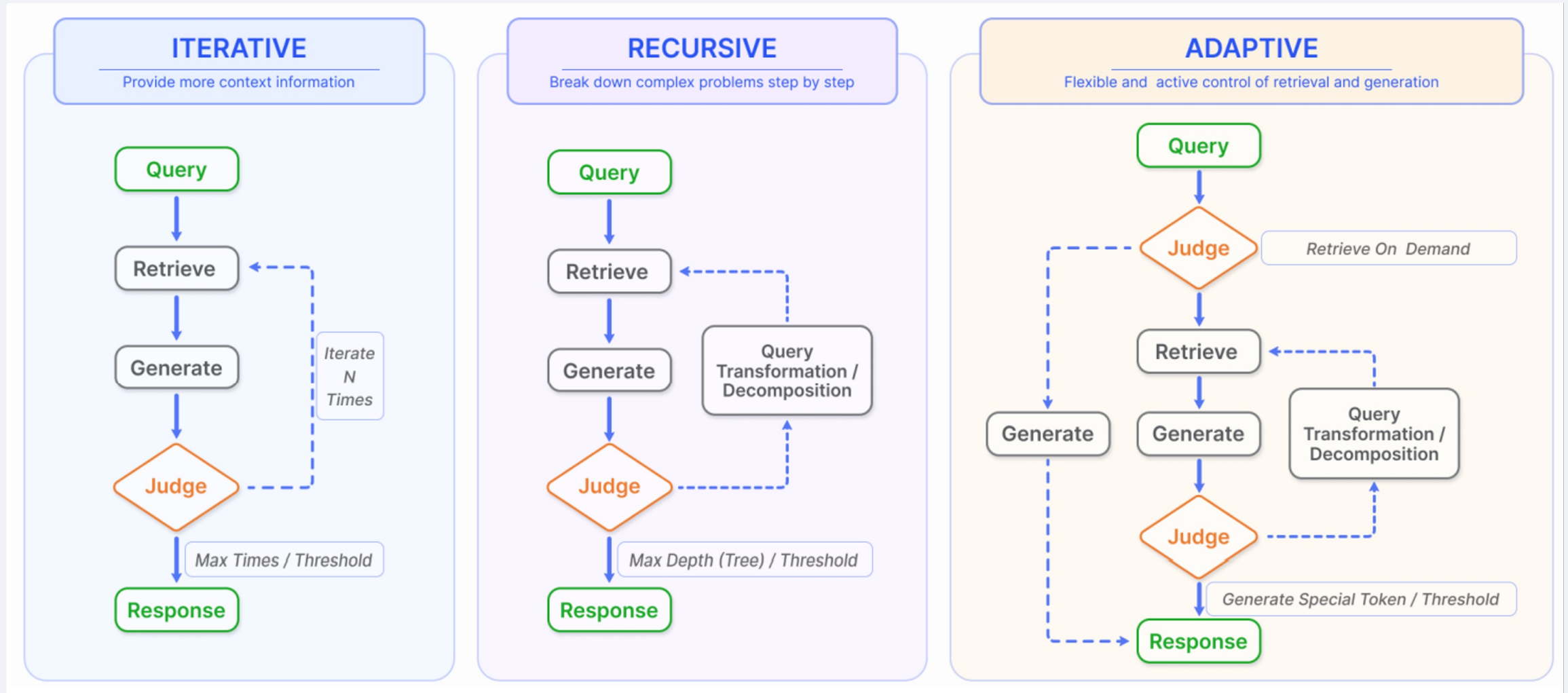
Retrieval Augmented Generation (RAG)

The RAG pipeline can be optimized at different levels.

- Document chunking
- Rewriting or extending the query
- Reranking with a larger model
- Analysis of the generator's response and regeneration or search for missing documents



Retrieval Augmented Generation (RAG)



Multiagent systems – example

A RAG pipeline where each component (query transformation, document retrieval, selection, generation) is treated as an agent with a separate reward function. It is optimized with reinforcement learning, maximizing the shared end reward.

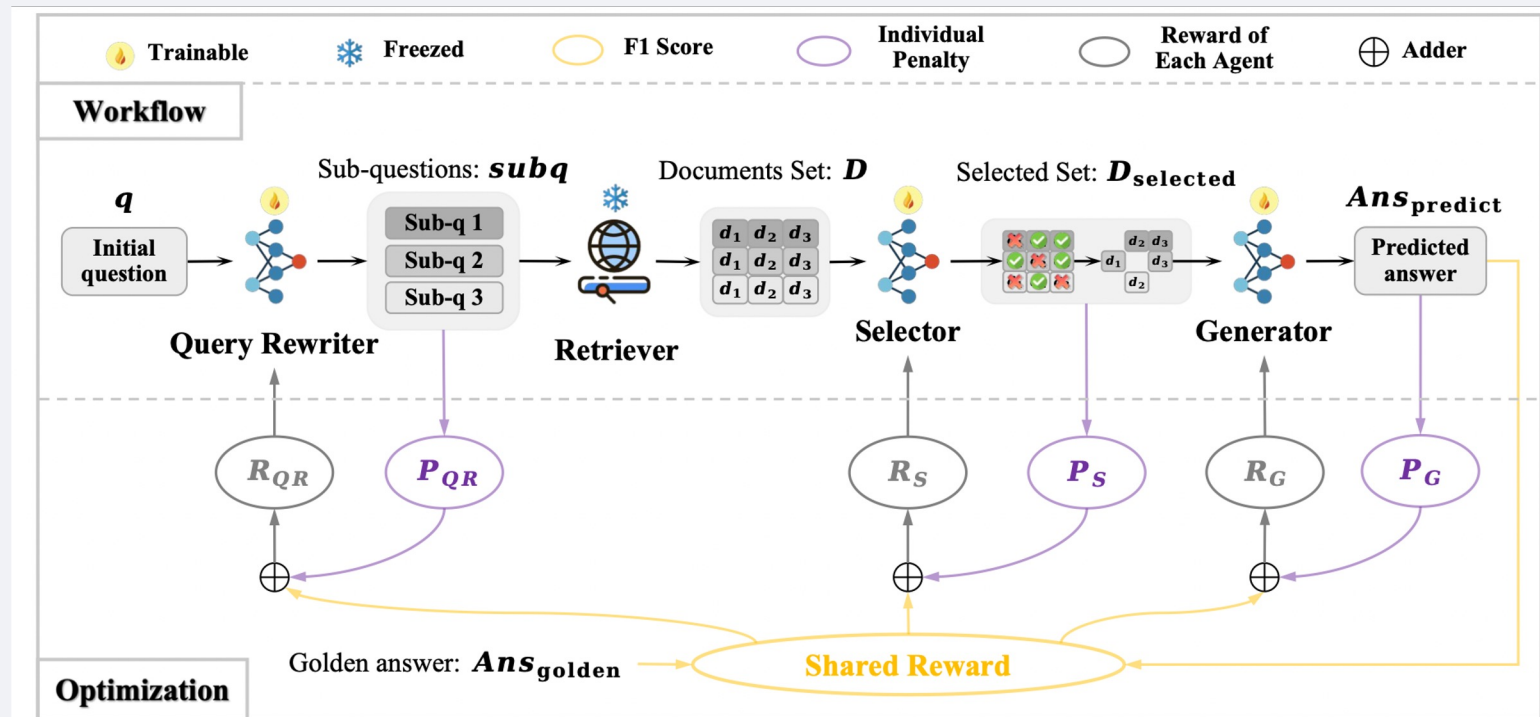
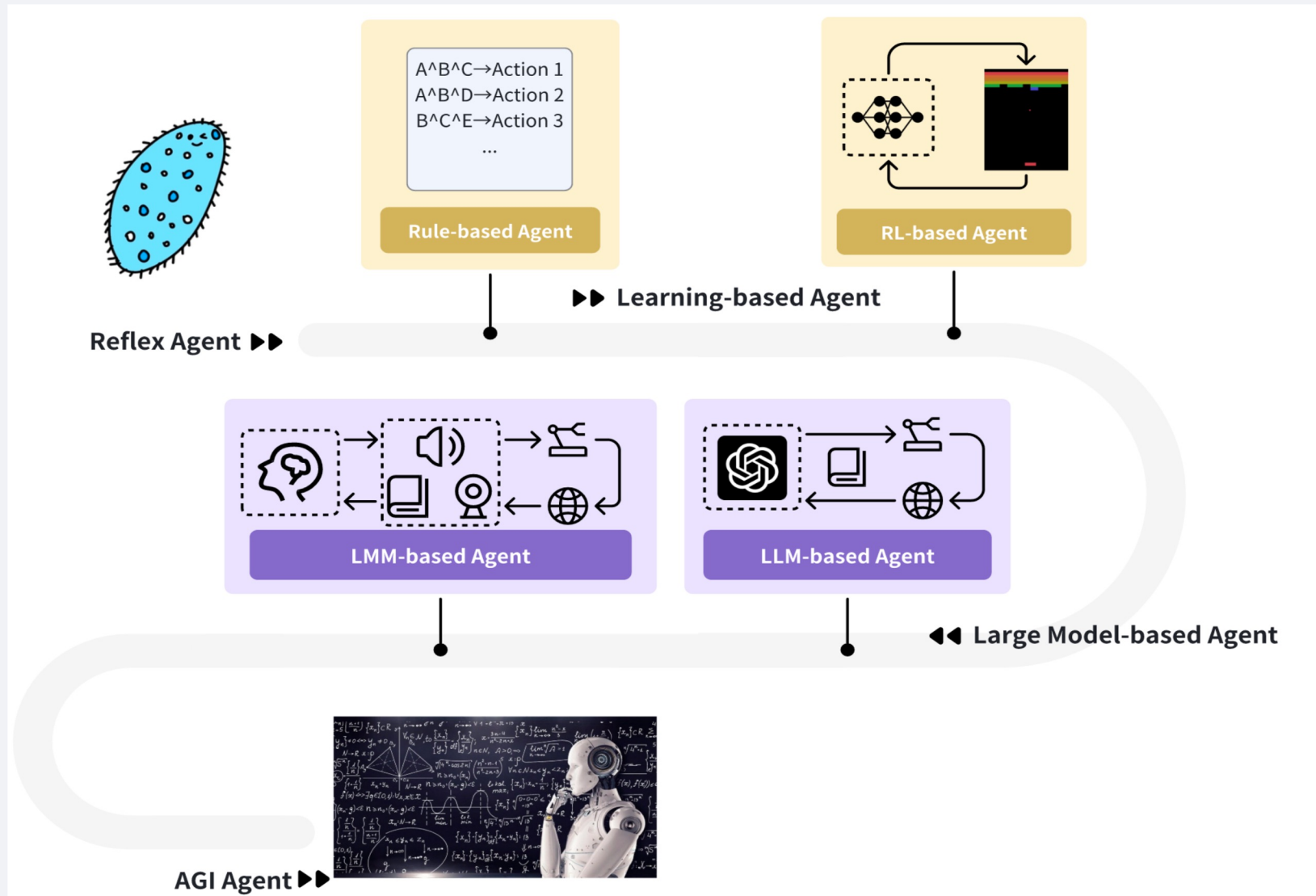


Figure 1: The overall framework of MMOA-RAG.

Agents



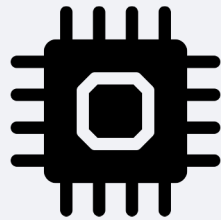
LLM-based Agents

An agent is a modular structure in which an LLM autonomously analyses a problem, makes decisions about actions, and executes them.



(1) Environment

Decision-making space



(2) Memory

short-term, long-term



(3) Tools

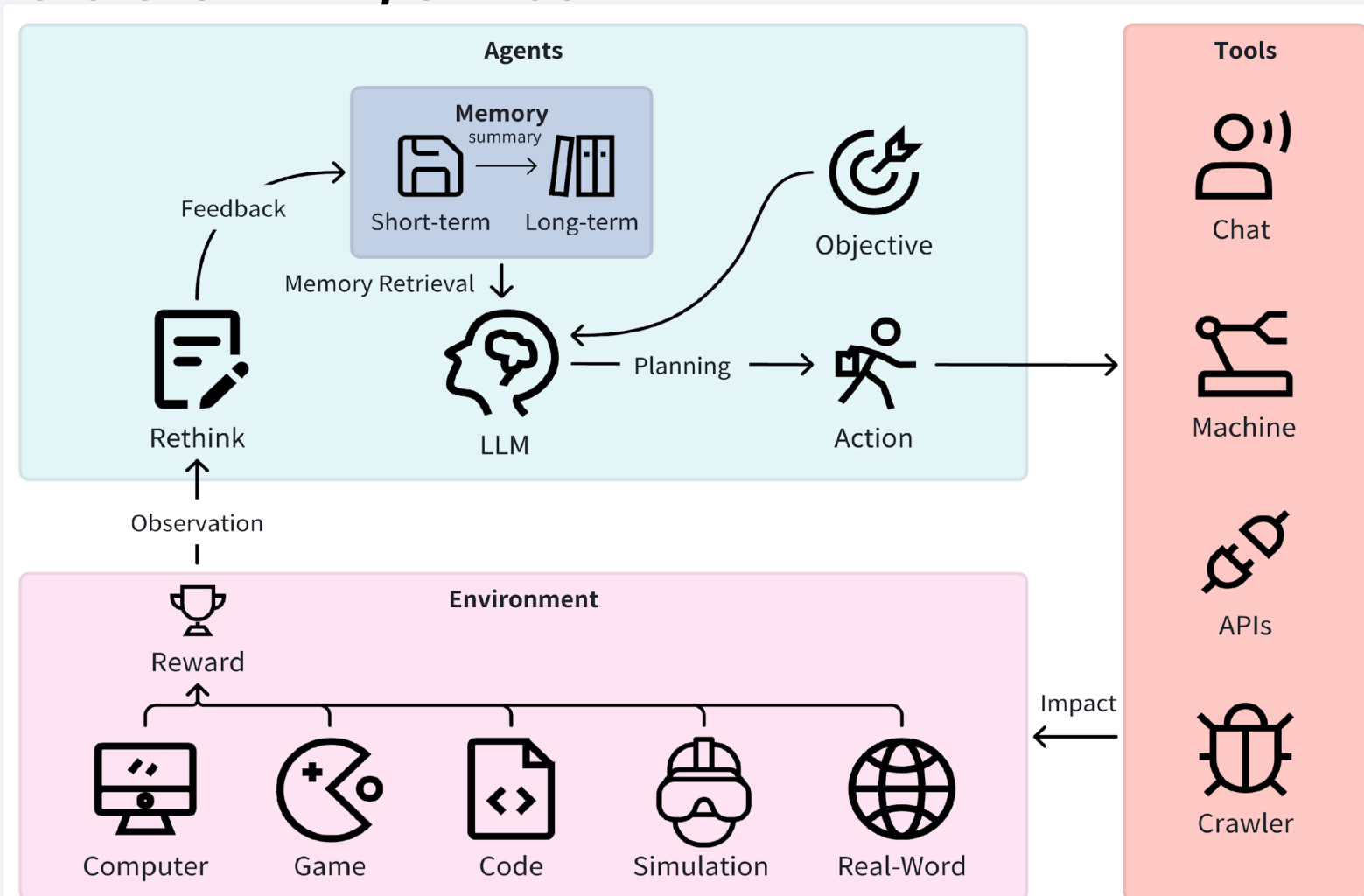
*Function Calling, web
search, API calls*



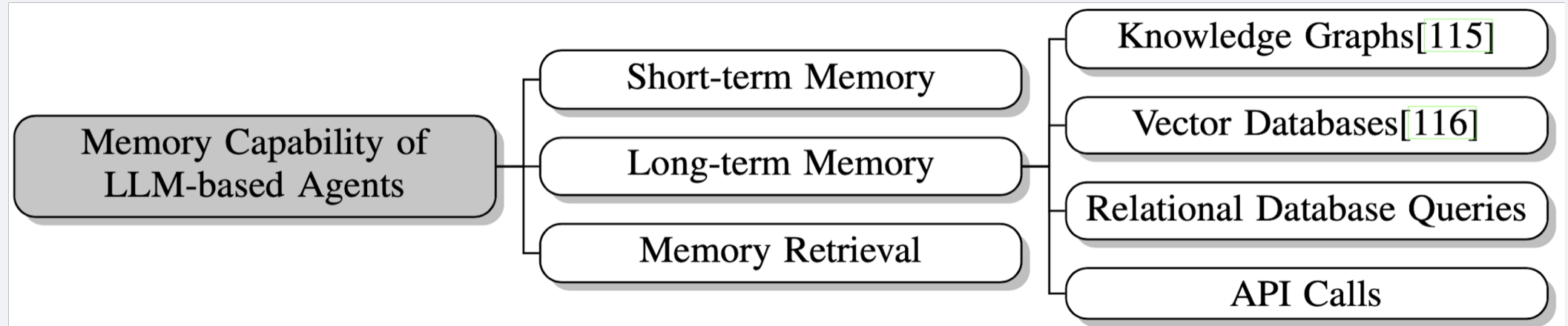
(4) Planning and reasoning

*Decomposition of tasks, self-
reflection*

LLM-based Agents



(2) Memory



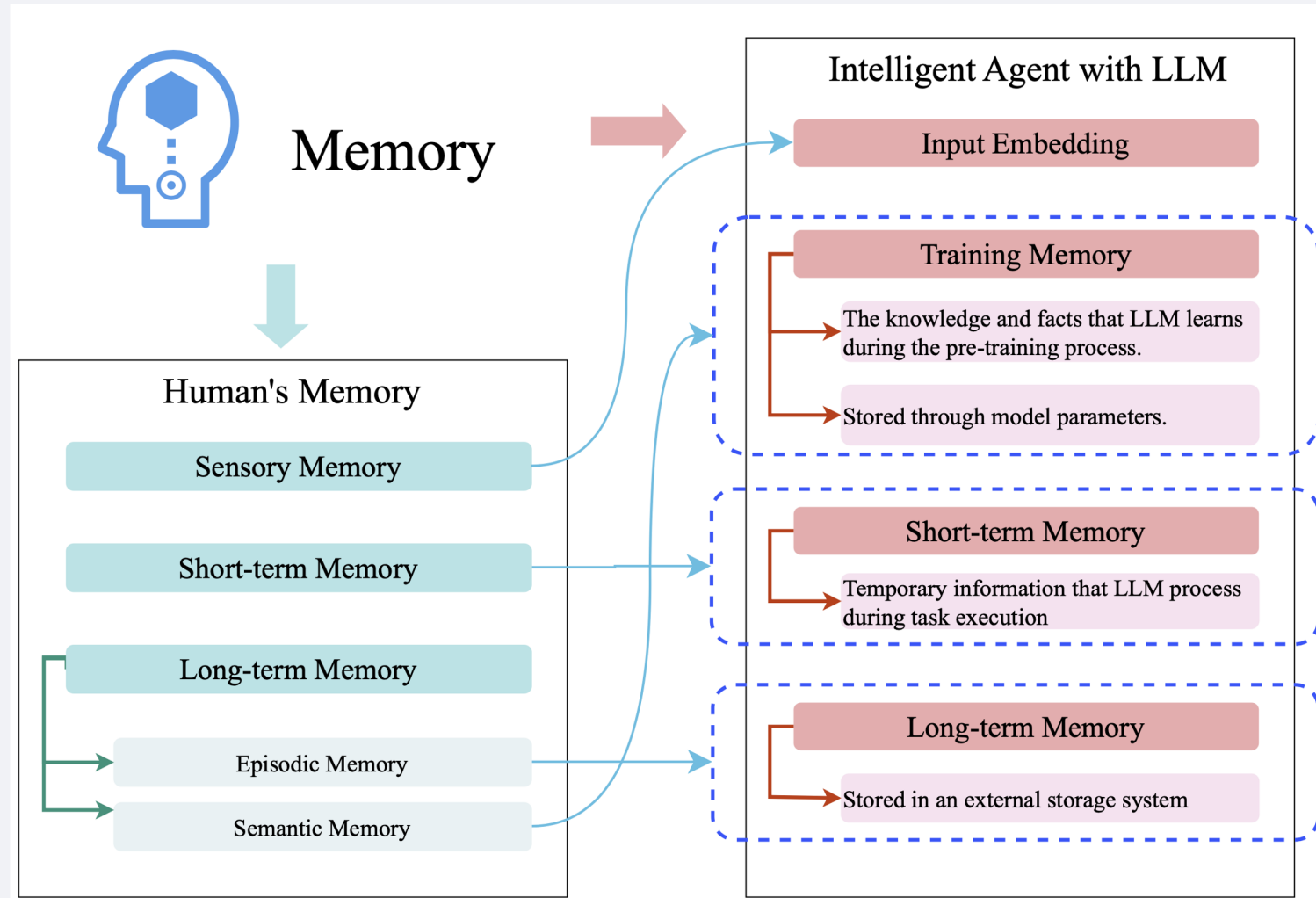
- Short-term memory - stores the current context and recent activities of the agent
 - prompt context
 - History of dialogue (memory buffer)
 - Context compression (through summarisation, distillation)

"LangChain [77] enhances short-term memory efficiency by encapsulating crucial information from each interaction and preserving the most recurrent interactions"

(2) Memory

- Long-term memory - allows you to store important information between interactions (data about earlier stages of activity, user, ...)
 - knowledge bases and file-based memory repositories (for example, memory pools)
 - vector storage of episodes – vector stores
 - Save state between sessions – memory persistence
- Memory Retrieval – searching and extracting knowledge from external sources
 - Sourcing decisions, e.g. classifier,
 - Strategies for Preservation (Remembering) and Pruning (Forgetting).
 - Retrieval Augmented Generation

(2) Memory



(3) Function Calling

- Allows the agent to perform actions outside the model: functions, APIs, databases, external programs
- It allows the agent to act procedurally, not just generate text
- Scheme:

recognizing the need to use the tool → calling the function → getting the result → analysing the result → deciding on the next action

- Supervised fine-tuning on examples of tool use
- Challenges: recognising when the tool is used, correcting the parameters of the functions/tools called

(3) Function Calling

We provide the model with a JSON schema for the available functions → The model selects the function with arguments → The environment calls the appropriate tool.

Example of data used for supervised learning:

```
SYSTEM: You are a helpful assistant with access to the following functions. Use them if required -  
{  
  "name": "get_news_headlines",  
  "description": "Get the latest news headlines",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "country": {  
        "type": "string",  
        "description": "The country for which to fetch news"  
      }  
    },  
    "required": [  
      "country"  
    ]  
  }  
}
```

```
USER: Can you tell me the latest news headlines for the United States?
```

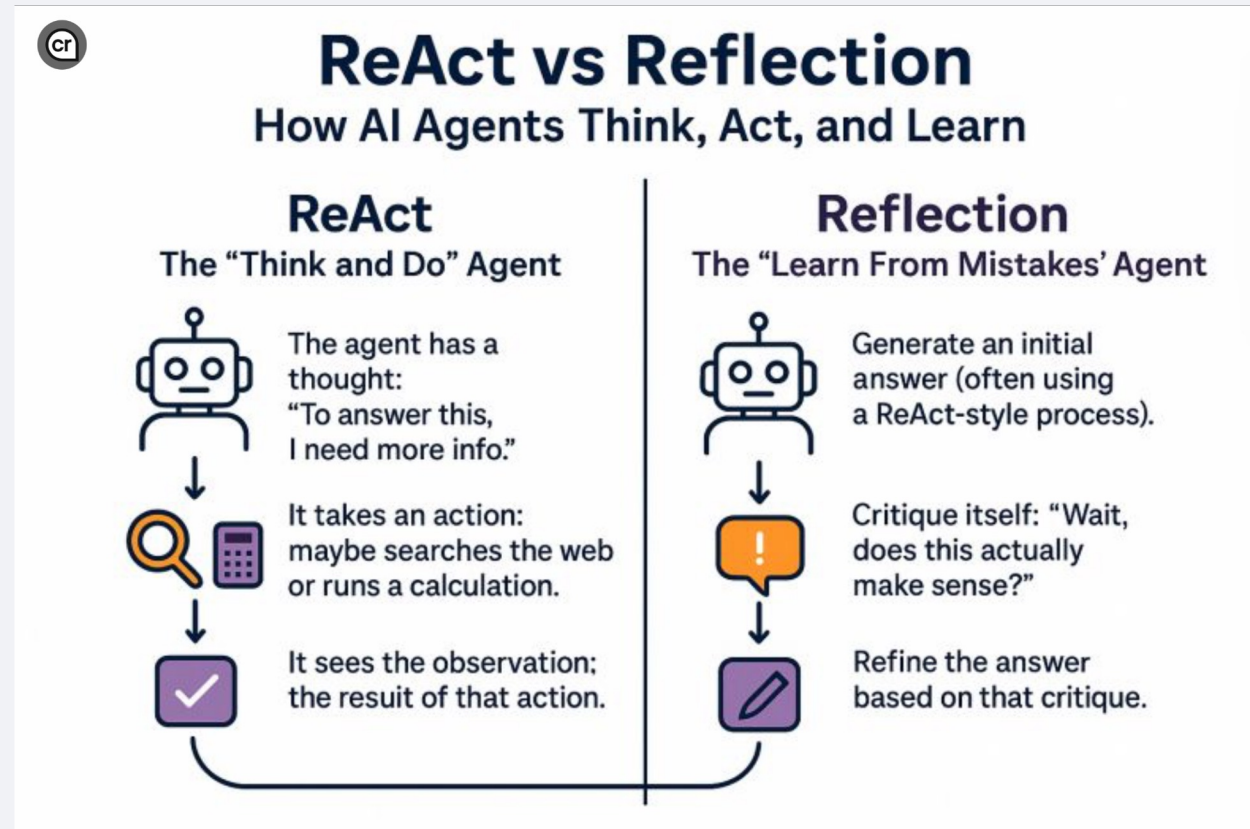
```
ASSISTANT: <functioncall> {"name": "get_news_headlines",  
  "arguments": '{"country": "United States"}'} <|endoftext|>
```

```
FUNCTION RESPONSE: {"headlines": ["Biden announces new vaccine mandates", "Hurricane Ida devastates Louisiana", "Apple unveils new iPhone", "NASA's Perseverance rover collects first Mars rock sample"]}
```

```
ASSISTANT: Here are the latest news headlines for the United States:  
1. Biden announces new vaccine mandates  
2. Hurricane Ida devastates Louisiana  
3. Apple unveils new iPhone  
4. NASA's Perseverance rover collects first Mars rock sample  
<|endoftext|>
```

(4) Planning and reasoning

- *In-context learning* – a cycle: generate → evaluate → correct. Enables plan adaptation during generation (e.g. ReAct (Yao et al. 2023), Reflexion (Shinn et al. 2023), CoT)



(4) Planing and reasoning

- Supervised methods and reinforcement learning – teach the agent (de facto LLM) to better plan or correct logic through information from the environment or giving a pattern (ground truth):
 - Supervised: correct steps and plan structure (np. Process Supervision, Hinting),
 - Enhancing: Improving strategies with feedback from the environment (e.g. *Dialogue Shaping*, *REX*)
- Multimodule and multiagent systems – combining separate planning, memory, and execution modules (e.g. *architektury planner–executor*)

(4) Reasoning – example: gsm8k

System prompt

You are an AI assistant that uses a Chain of Thought (CoT) approach with reflection to answer queries.

Follow these steps:

- Think through the problem step by step within the `<thinking>` tags.
- Reflect on your thinking to check for any errors or improvements within the `<reflection>` tags.
- Make any necessary adjustments based on your reflection.
- Provide your final, concise answer within the `<output>` tags.

Important: The `<thinking>` and `<reflection>` sections are for your internal reasoning process only.

Do not include any part of the final answer in these sections.

The actual response to the query must be entirely contained within the `<output>` tags.

Use the following format for your response:

`<thinking>`

[Your initial thought process goes here]

`</thinking>`

`<reasoning>`

[Your step-by-step reasoning goes here. This is your internal thought process, not the final answer. You can create as many reasoning steps as necessary in your process.]

`</reasoning>`

`<reflection>`

[Your reflection on your reasoning, checking for errors or improvements. You can create as many reflection steps as necessary in your process.]

`</ reflection>`

`<adjustment>`

[Any adjustments to your thinking based on your reflection]

`</adjustment>`

`<output>`

[Your final, concise answer to the query. This is the only part that will be shown to the user.]

`</output></thinking>`

(4) Reasoning – example: gsm8k

Question

A craft store makes a third of its sales in the fabric section, a quarter of its sales in the jewelry section, and the rest in the stationery section. They made 36 sales today. How many sales were in the stationery section?

Answer

Thus, there were $36 - 12 - 9 = \ll 36 - 12 - 9 = 15 \gg 15$ sales in the stationery section.

(4) Reasoning – example: gsm8k

Generation

<thinking>

To find the number of sales in the stationery section, I need to first calculate the number of sales in the fabric and jewelry sections. The store makes a third of its sales in the fabric section, which is $\frac{1}{3}$ of 36 sales. I can calculate this as $36 * \frac{1}{3} = 12$ sales in the fabric section. The store makes a quarter of its sales in the jewelry section, which is $\frac{1}{4}$ of 36 sales. I can calculate this as $36 * \frac{1}{4} = 9$ sales in the jewelry section. Since the rest of the sales are in the stationery section, I can subtract the sales from the other two sections from the total number of sales to find the number of sales in the stationery section.

</thinking>

<reasoning>

1. Calculate the number of sales in the fabric section: $36 * \frac{1}{3} = 12$

2. Calculate the number of sales in the jewelry section: $36 * \frac{1}{4} = 9$

3. Calculate the number of sales in the stationery section: $36 - 12 - 9 = 15$

</reasoning>

<reflection>

Upon reviewing my reasoning, I noticed that my calculations seem correct, but I should ensure that I understand the question correctly. The question asks for the number of sales in the stationery section, and I believe I have correctly calculated that. However, I should double-check my math to make sure it's accurate.

</reflection>

<adjustment>

Upon re-examining my math, I am confident that my calculations are correct. The fabric section makes up $\frac{1}{3}$ of the sales, the jewelry section makes up $\frac{1}{4}$, and the rest is the stationery section. My math checks out, and I am confident in my answer.

</adjustment>

<output>

15

</output>

(4) Reasoning – example: gsm8k

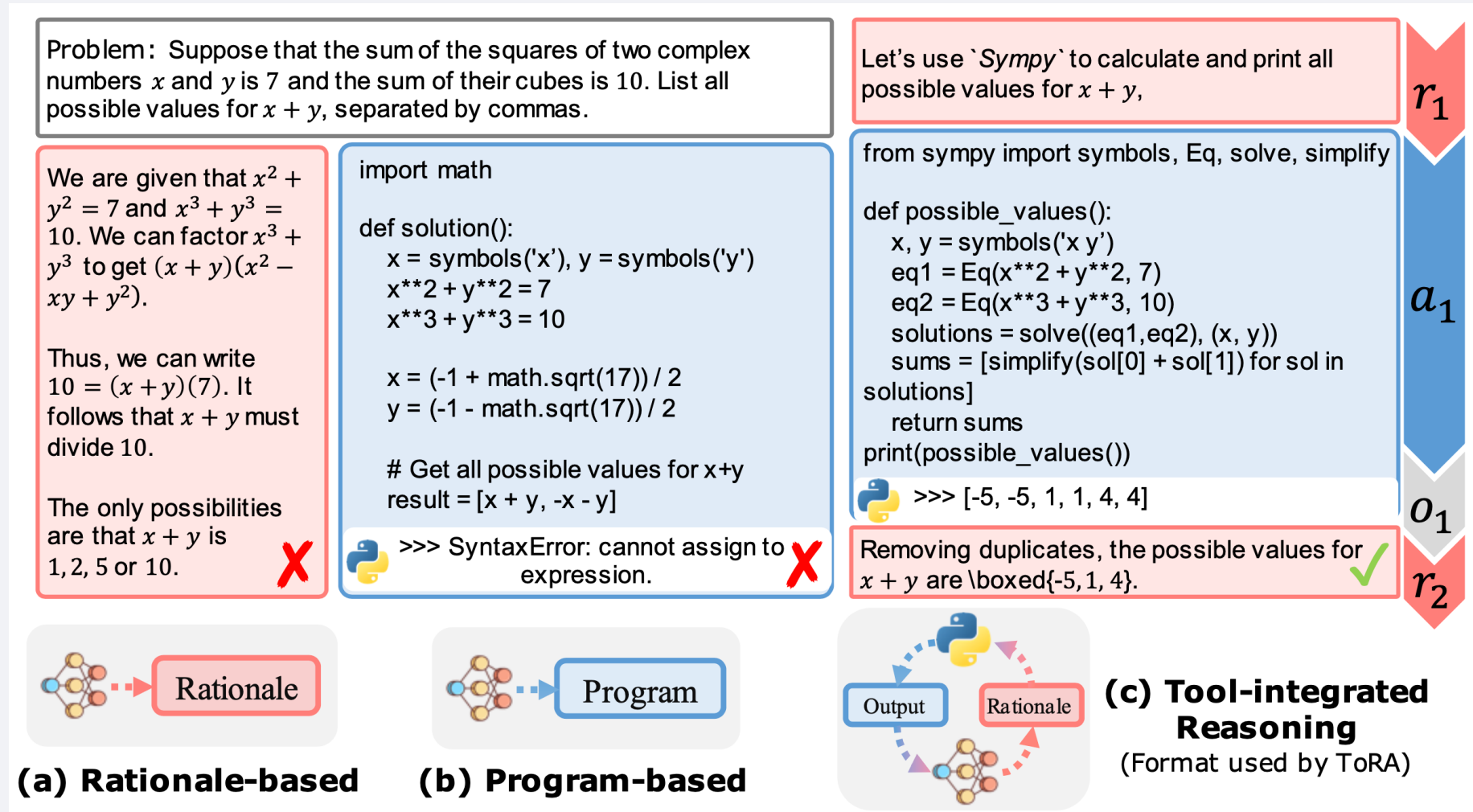
Messages (*course of action*)

```
[
  {
    "content": system prompt
    "role": "system"
  },
  {
    "content": task,
    "role": "user"
  },
  <{
    "content": recording of the generated output,
    "role": "assistant"
  }
]
```

TORA – tool intergration and reasoning

- ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving (Gou et al., 2024)
- Chain of Thought
 - Chain of Thought
 - PAL: Program-aided Language Models (Gao et al., 2023)
 - combination of reasoning and complex tool use via program code
 - universality of the solution, precision and flexibility of the combination
- Training process
 - knowledge distillation + remotely (weakly) supervised training of an LLM

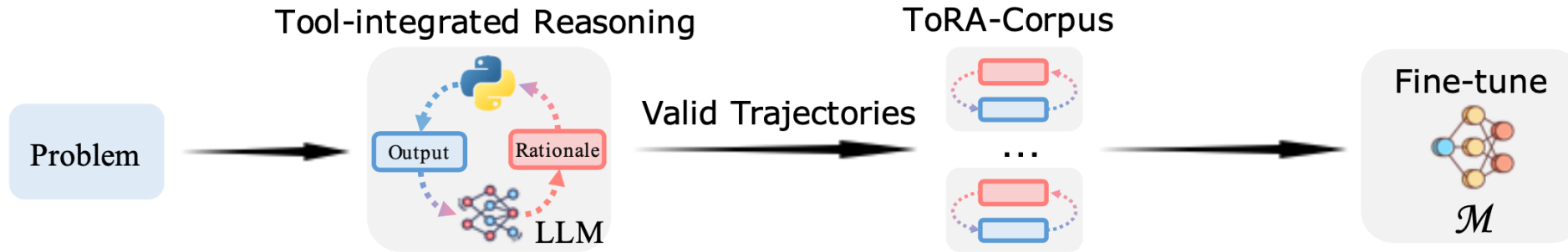
TORA – reasoning schemas



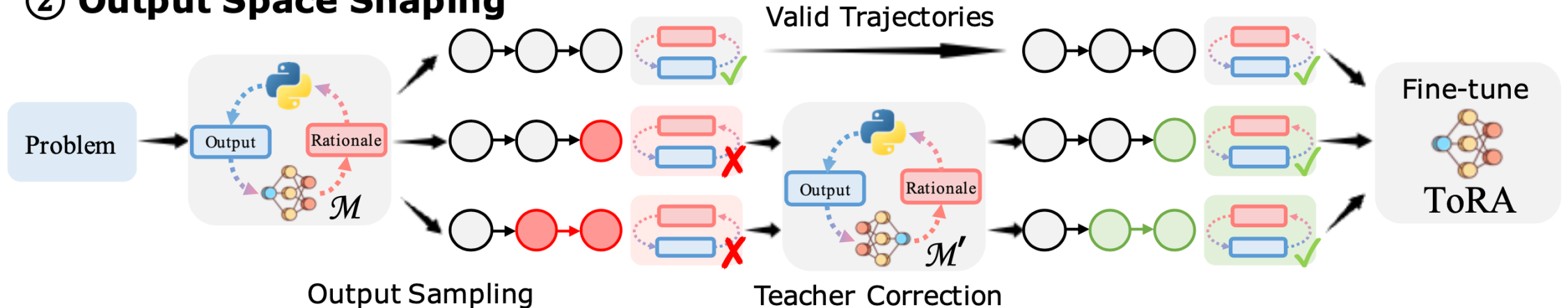
- Tool integration as a combination of Chain of Thought and Program-based

TORA – two stage training

① Imitation Learning



② Output Space Shaping



- Imitation learning on the basis of simulation by strong LLM
- Remotely (weakly) supervised finetuning

TORA – reasoning example

Question: Solve the inequality

$$\left[\frac{x^2 - 25}{x + 5} < 0.\right]$$

Solution:

Let's use 'sympy' to solve the inequality:

```
“python
```

```
from sympy import symbols, simplify
```

```
from sympy.solvers.inequalities import solve_univariate_inequality
```

```
from sympy.core.relational import StrictLessThan
```

TORA – reasoning example

```
def solve_inequality():
    x = symbols('x')
    expression = (x**2 - 25) / (x + 5)
    inequality = StrictLessThan(expression, 0)
    solution = solve_univariate_inequality(inequality, x, relational=False
    )
    simplified_solution = simplify(solution)
    return simplified_solution
result = solve_inequality()
print(result)
““
““output
Union(Interval.open(-oo, -5), Interval.open(-5, 5))
““
The answer is  $\boxed{(-\infty, -5) \cup (-5, 5)}$ 
```

TORA – reasoning example

Question: What real values of x are not in the domain of $f(x) = \frac{1}{|x^2 - 6x + 8| + |x^2 + x - 6|}$?

Solution:

The domain of the function is all real numbers, so we need to find the values of x for which the denominator of the function is zero.

The denominator of the function is the sum of the absolute values of the two terms inside the absolute value.

Each of these terms is a quadratic in x , and the roots of a quadratic equation are real unless the equation is identically zero.

So, we need to find the values of x for which the two quadratic equations are identically zero.

“python

```
from sympy import symbols, Abs, solveset, S
```

```
from sympy.core.relational import Ne
```

TORA – reasoning example

```
def non_domain_values():
    x = symbols('x')

    # These are the values of x for which the function is not defined
    roots1 = solveset(Ne(x**2 - 6*x + 8, 0), x, domain=S.Reals)
    roots2 = solveset(Ne(x**2 + x - 6, 0), x, domain=S.Reals)

    # The roots of a quadratic equation are always in the domain of the
    function, so we need to find the values of x that are not in the
    union of the roots
    non_domain_values = S.Reals - (roots1.union(roots2))

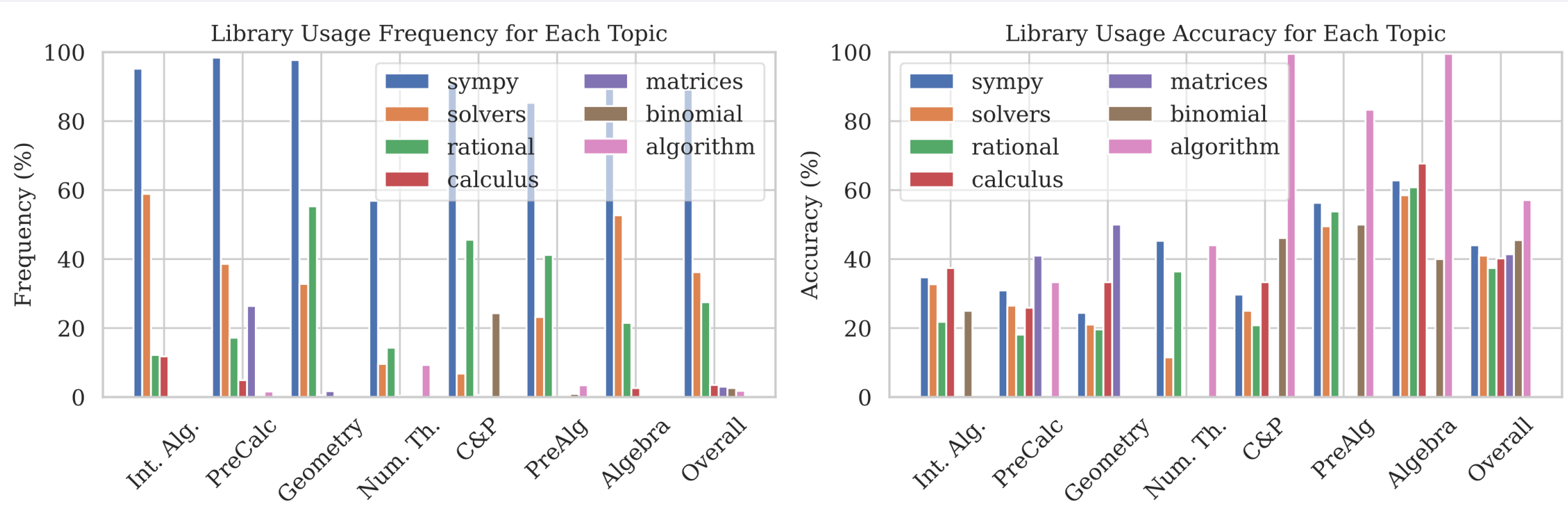
    return non_domain_values

result = non_domain_values()
print(result)

““
““output
{2}
““

The real value of  $x$  that is not in the domain of the function is  $\backslash$ 
boxed{2}$.
```

TORA – types of the applied tools



- sympy – parsing of symbolic expressions, solvers – algebraic constraints problem solving,
- matrices – operations on matrices, algorithms – numerical optimisation, rational – geometric calculations for factorial equations

Additional Literature

- Jimmy Lin, Rodrigo Nogueira and Andrew Yates. Pretrained Transformers for Text Ranking: BERT and Beyond. arXiv: 2010.06467v3, cs.IR, 2021.
- Vaibhav Mavi, Anubhav Jangra i Adam Jatowt. A Survey on Multi-hop Question Answering and Generation. arXiv:2204.09140v1, 2022.
<https://doi.org/10.1145/1122445.1122456>
- Semih Yavuz, Kazuma Hashimoto, Yingbo Zhou, Nitish Shirish Keskar i Caiming Xiong. Modeling Multi-hop Question Answering as Single Sequence Prediction. Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics Volume 1: Long Papers, pages 974 – 990, May 22-27, 2022, 2022 Association for Computational Linguistics. <https://aclanthology.org/2022.acl-long.69.pdf>

Additional Literature

- Yunfan Gao et al. (2024) Retrieval-Augmented Generation for Large Language Models: A Survey. <https://arxiv.org/pdf/2312.10997>
- ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT (<https://arxiv.org/abs/2004.12832>)
- Jina-ColBERT-v2: A General-Purpose Multilingual Late Interaction Retriever (<https://arxiv.org/abs/2408.16672>)
- MAKING TEXT EMBEDDERS FEW-SHOT LEARNERS (<https://arxiv.org/pdf/2409.15700>)

Additional Literature

1. Yiu L., Kilicoglu H., [Commonsense-Aware Prompting for Controllable Empathetic Dialogue Generation](#), 2023
1. Madaan A. et al., [Language Models of Code are Few-Shot Commonsense Learners](#), 2023
2. Lu Y. et al., [Neuro-Symbolic Procedural Planning with Commonsense Prompting](#), 2023